

Artificial Intelligence - Project 1

Part A - Requirement R1: Pseudocode

Name: Harsana Neupane

Overview & Parameters

- N = 60 nodes (city locations)
- Initial p = 0.05 (increments by 0.01 if graph is not fully connected)
- Existing road lengths = between 5 and 25
- Simulation time = 10 hours (36,000 seconds/ticks)
- T = 100 random trips per second
- Shrinkage factor f = 0.6
- Budget k = max number of roads to build

Main Algorithm

```
Algorithm RoadPlanSimulation:
    // Step 1: make network and trip matrix
    network = createConnectedNetwork(60, 0.05)
    tripMatrix = array [60][60] filled with 0

    // Step 2: Run traffic simulation
    collectTrafficData(network, tripMatrix, 100)

    // Step 3: Calculate benefit for unconnected pairs
    benefits = computeAllBenefits(network, tripMatrix, 0.6)

    // Step 4: choose roads until k roads are built
    built = 0
    while built < k:
        best = getMostBenefitRoad(benefits)

        // if no candidate roads remain
        if best is null:
            break

        L, M = best
        print("Recommended road to build:", L, M)

        addDirectRoad(network, L, M, 0.6)
        benefits[L][M] = -1
        benefits[M][L] = -1

        // nearby values change
        updateMatrixAfterRoadBuilt(network, tripMatrix, benefits, L, M, 0.6)

        built = built + 1
```

Helper Functions

```
Function createConnectedNetwork(N, initialp):
    p = initialp

    repeat:
        G = generateRandomGraph(N, p)

        // assign road lengths to existing edges
        for each edge (u, v) in G:
            G.weight[u][v] = random_range(5, 25)

        if G is connected:
            return G
        else:
            p = p + 0.01    // increase if graph did not connect
```

```
Procedure collectTrafficData(network, tripMatrix, tripsPerTick):
```

```

for tick = 1 to 36000:
    for i = 1 to tripsPerTick:
        start = randomNode(0, 59)
        end = randomNode(0, 59)

        // make start and end different
        while end == start:
            end = randomNode(0, 59)

        // use A* to find shortest path
        path = A_Star_Search(network, start, end)

        tripMatrix[start][end] = tripMatrix[start][end] + 1

Function calculateSingleBenefit(X, Y, network, tripMatrix, f):
    shortXY = A_Star_Distance(network, X, Y)
    d_XY = f * shortXY

    directTrips = tripMatrix[X][Y] + tripMatrix[Y][X]
    benefit = (shortXY - d_XY) * directTrips

    for each n1 in Neighbors(network, Y):
        currentDist = A_Star_Distance(network, X, n1)
        savings = currentDist - d_XY - getDistance(network, Y, n1)

        if savings > 0:
            trips = tripMatrix[X][n1] + tripMatrix[n1][X]
            benefit = benefit + (savings * trips)

    for each n2 in Neighbors(network, X):
        currentDist = A_Star_Distance(network, Y, n2)
        savings = currentDist - d_XY - getDistance(network, X, n2)

        if savings > 0:
            trips = tripMatrix[Y][n2] + tripMatrix[n2][Y]
            benefit = benefit + (savings * trips)

    return benefit

Function computeAllBenefits(network, tripMatrix, f):
    benefitMatrix = matrix of size [60][60] set to -1

    for X = 0 to 58:
        for Y = X + 1 to 59:
            if not isDirectlyConnected(network, X, Y):
                val = calculateSingleBenefit(X, Y, network, tripMatrix, f)
                benefitMatrix[X][Y] = val
                benefitMatrix[Y][X] = val

    return benefitMatrix

Procedure updateMatrixAfterRoadBuilt(network, tripMatrix, benefitMatrix, L, M, f):
    // As mentioned in Tutorial 1, building a road affects nearest neighbors
    // of endpoints L and M.
    affectedNodes = getNeighbors(network, L) + getNeighbors(network, M)

    for each node in affectedNodes:
        for otherNode = 0 to 59:
            if node != otherNode and
               not isDirectlyConnected(network, node, otherNode):

                if benefitMatrix[node][otherNode] != -1:
                    // skip already built roads
                    newVal = calculateSingleBenefit(
                        node, otherNode, network, tripMatrix, f
                    )

                    benefitMatrix[node][otherNode] = newVal
                    benefitMatrix[otherNode][node] = newVal

```